CE152: Object-Oriented Programming

Conditionals

Dr. Sebastian Halder

CSEE

2023

Refresher operators

Comparison operators

- There are several other operators we could use in Java

Operator	Meaning	true	false
==	equal	4 == 4	4 == 5
!=	unequal	4 != 5	4 != 4
<	less than	4 < 5	4 < 4
<=	less than or equal	4 <= 5	5 <= 4
>	greater than	5 > 4	5 > 5
>=	greater than or equal	5 >= 5	4 >= 5

Logical operators

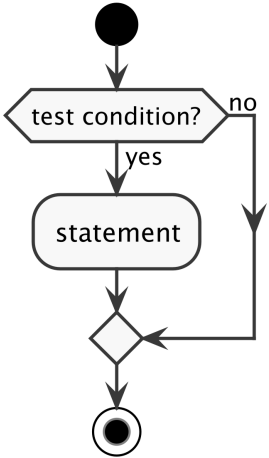
- If we wanted to have two conditions we can combine statements with a logical operator:
  - Logical *and*: &&
  - Logical *or*: ||
  - Logical *not*: !
- We often use operators in conditional statements to decide which segment of code to execute during runtime

Conditionals

Using if statements

- An *if* statement executes the if-branch of the statement if the condition is true (we do this to execute this part of code only under those conditions)
- An *if* combined with an *else* statement executes either the if-branch (condition true) or the else-branch (condition false)
- We can use a combined *else if* to make the else statement conditional (see example later)

if UML



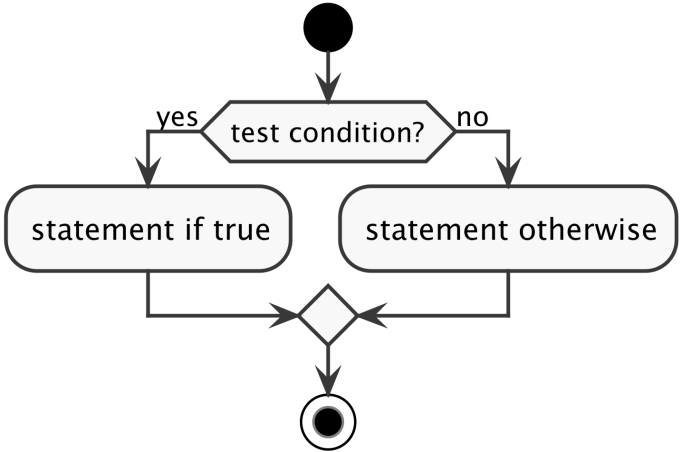
Example if

```
if (x < 0) {
  System.out.println("x is negative.");
}

if (x > 0) {
  System.out.println("x is positive.");
}
```

- Note: the {} are not needed if the segment of code following the `if` is only one line of code

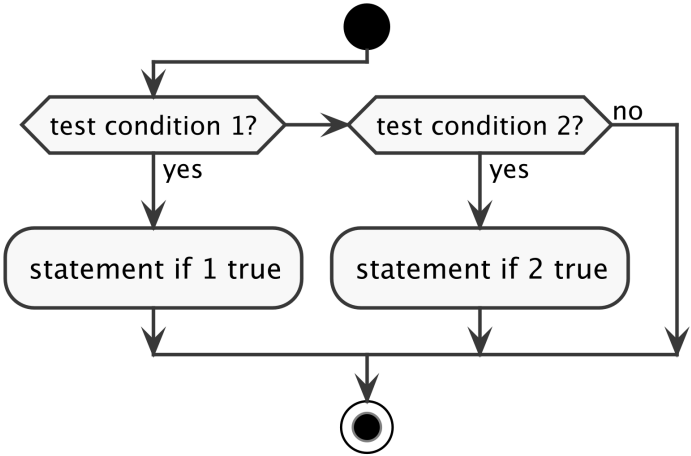
if-else UML


yes
test condition?
no
statement if true
statement otherwise

Example if-else

```
if (x < 0) {
  System.out.println("x is negative.");
}
else {
  System.out.println("x is positive or zero.");
}
```

if-else-if UML



Example if-else-if

```
if (x < 0) {
  System.out.println("x is negative.");
}
else if (x > 0){
  System.out.println("x is positive.");
}
```

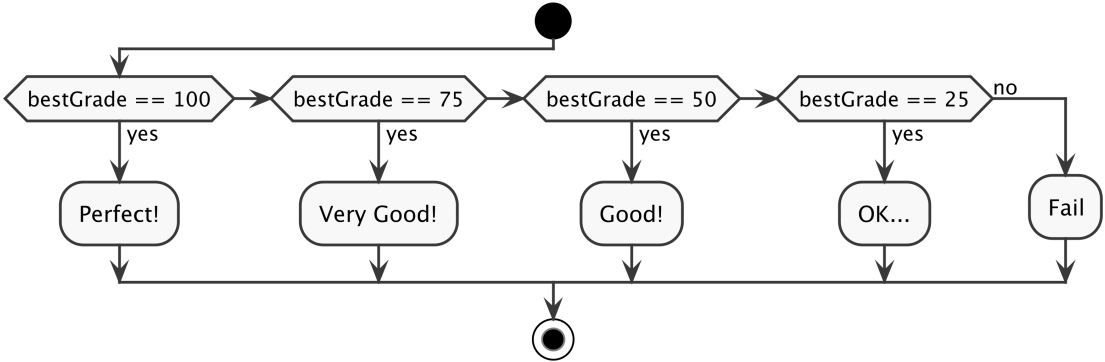
- We could add another `else`:

```
else {
  System.out.println("x is zero.");
}
```

Feedback of the best grade

- We want a Lecturer to provide feedback after grading the exam
- The Lecturer should provide the feedback based on the percentage of correct answers

Defining grade feedback UML



- Disclaimer: these are not the real degree classes

Defining the feedback Java

```
String degreeClass = "FAIL";

if (bestGrade == 100) degreeClass = "Perfect!";
else if (bestGrade == 75) degreeClass = "Very Good!";
else if (bestGrade == 50) degreeClass = "Good!";
else if (bestGrade == 25) degreeClass = "OK";
else                      degreeClass = "FAIL";
//do we need the last else?

System.out.println("The grade is: " + degreeClass);
```

The conditional operator

- This is a special syntax for defining a `if-then-else` statement:
- `variable x = (expression) ? value1 if true : value2 if false`
- Meaning: if the `expression` is true assign `value1` to `x` otherwise assign `value2` to `x`
- Many languages provide this syntax because it allows shorter code
- Examples:

```
int x = 5;
// if x is greater than zero assign 1 to y else assign -1
int y = (x > 0) ? 1 : -1;
```

Using a conditional operator

- We could rewrite the group of if-else statements:

```
degreeClass = bestGrade == 100 ? "Perfect!":
              bestGrade == 75 ? "Very Good!":
              bestGrade == 50 ? "Good!":
              bestGrade == 25 ? "OK":
              "FAIL";
```

Streamlining with conditional operators

- As you can see the amount of code was reduced
- We did not have to set the variable *degreeClass* multiple times (we avoided duplication)
- Generally making programs more concise in this way improves readability
- Nevertheless you should not overdo this

Comparing Objects

- The logical operators do not function as we expect on Objects

```
String s1 = new String("CE152");
String s2 = new String("CE152");
System.out.println(s1==s2); // this will be false
System.out.println(s1.equals(s2)); // this will be true
```

- This is because the == operator compares the hashcode of the Objects

String comparisons

```java
String s1 = "oop";
String s2 = "oop";
System.out.println(s1==s2);
// this will be TRUE because Java uses the existing Object

String s3 = new String("oop");
String s4 = new String("oop");
System.out.println(s3 == s4);
// this will be FALSE because we manually create a new Object
```

Live coding

- Examples of `if` and `else` statements using `Math.random`
- Example of conditional operator
- Comparison operators using primitves and Objects

Summary

Conditionals

- Conditionals exist in many programming languages
- They allow us to execute different segments of code depending on the value of certain variables
- We used:
  - `if` : segment is executed if the condition is true
  - `else`: follows an `if` and is always executed if the `if` statement is false
  - `else if`: also follows an `if` but is only executed if its own statement is true
- We also used a special syntax for defining `if-then-else` statements using the conditional operator:
  - `variable x = (expression) ? value1 if true : value2 if false`
- Remember Objects are not compared based on the content but the memory address
- Use dedicated methods such as `equals()` when comparing Objects

Thank you and see you in the next video!